ALBUKHARY INTERNATIONAL UNIVERSITY

DU014 (K)

CRYPTOGRAPHY ESSENTIALS

PROJECT PROPOSAL

AES-Based Secure File Encryption and Decryption Tool

Student Name	_____Maralbyek Tilyek_____
Student ID	_____AIU24102280_____
Course / Course Code	Cryptography Essentials CCC2243
Project Type	Individual Project
Year / Semester	Year 2, Semester 3, 2025-2026

Table of Content

1. Background

Information found in digital files regularly has actual value to someone other than the file's owner: academic transcripts, financial statements, medical records, source code or personal identification documents. If protected files are not stored on a protected laptop, removable drive, or shared network location, anyone who has physical or remote access to the storage medium can read, copy, and modify the files as they see fit. The issue of exposure is directly attacked in encryption by converting the readable plaintext into ciphertext that is difficult to decrypt without the secret key.
Advanced Encryption Standard (AES): A public, multi-year evaluation of candidate ciphers led by the U.S. National Institute of Standards and Technology (NIST) to determine the most suitable symmetric-key block cipher for widespread adoption is complete and led to the adoption of AES in FIPS PUB 197 (2001). Since then it has been the de facto global standard for data protection at rest, which it backs to technologies from full-disk encryption (BitLocker, FileVault) to secure messaging and TLS. This project will use AES, in the authenticated Galois/Counter Mode (AES-GCM) variant, to create a usable end-user application, correctly implemented, that would be accessible to a non-technical user that could be operated using a graphical interface, bridging the gap between cryptographic theory and a usable application.

2. Problem Statement

Most of the people using computers every day do not use strong, standard encryption techniques for sensitive files: hiding a file in a hidden folder, using operating-system-level password prompts which do not encrypt the underlying bytes and use, or using consumer-grade archive software with weak or outdated ciphers as the default. None of these would offer any real confidentiality against an adversary who has direct access to the storage device, and few offer a way to determine if a file has been altered.

Meanwhile there are many applications and code samples readily available online that implement AES encryption incorrectly, such as by using Electronic Codebook (ECB) mode, which reveals structural patterns in plaintext; or using a direct password-based cryptographic key, rather than using a secure key-derivation function, which makes offline brute force attacks much more feasible; or without implementing any integrity/authentication mechanism, which means that an attacker can modify the ciphertext without it being detected when they decrypt it. It is perfectly clear that there is a need for a simple and correctly designed application that is easily available to every user and provides a safe, secure and correct implementation of AES encryption, which achieves confidentiality, immunity to brute force password attack and tamper resistance without the user being required to understand the cryptography behind the protection.

3. Aim

Create, execute and assess an intuitive and usable application for AES-256 (AES-GCM) password-based encryption/decryption of files with a password derived key, with confidentiality, tamper detection and password-based authentication in one convenient package.

4. Objectives

The specific objectives of this project are:

1. To investigate and write down the theory behind AES, symmetric-key cryptography, password-based key derivation (PBKDF2), authenticated encryption with associated data (AEAD), and the reasons behind each design.
2. Prior to implementation, it is essential to create a Use Case Diagram, Data Flow Diagram and a Flowchart that includes encryption and decryption paths to design the complete application workflow.
3. To implement secure key derivation from a user-supplied password, with a cryptographically random, per-file salt, and an iteration count corresponding to the current NIST/OWASP recommendations.
4. To use file encryption with AES-256-GCM, the random nonce used for each encryption operation must be different and the authentication tag produced by the encryption must be checked prior to any decrypted data being written to disk.
5. To create and implement a graphical user interface (Tkinter) that enables the selection of the file through a dialog, masks and confirms the password entry, provides a progress indicator for larger files and provides clear, non-technical success and failure messages.
6. To systematically test the tool with different file types and sizes, and with both correct and incorrect password, as well as with deliberately corrupted ciphertext, to check for both functional correctness and the ability of the tool to reject invalid decryption attempts.
7. To describe the design of the system, implementation decisions, threats model, testing, known limitations, and possible future enhancements in a final report suitable for academic evaluation.

5. Project Scope

5.1 In Scope

The tool will encrypt and decrypt files at the local level and in a single user. It will accept common file formats, such as text files, PDFs, images, and compressed files, while encrypting the raw bytes of the file, not just the text. All output files will be encrypted and will contain the required random salt, nonce, ciphertext and GCM authentication tag needed for subsequent decryption, without requiring a separate key file to be maintained by the user.

5.2 Out of Scope

- Uploading files to cloud or file transmission through the network.
- Multiple user account management, access control lists, or role-based permissions.
- By design: a forgotten password must make the file unrecoverable — this is a core security property, and not a limitation (password recovery mechanisms).
- Enterprise level key management, integration with hardware security modules (HSM) or centralized key escrow.
- Protection from attacks that need access to the running system's memory or an unlocked session (side-channel and cold-boot attacks).

6. Proposed Methodology

6.1 Encryption Workflow

1. The user uses a file-picker dialog box to choose a file and then enters a password to be checked for typos by a second entry field.
2. Using a secure random number generator, such as secrets in python or os.urandom in python, a cryptographic salt is generated to create a random 16-byte salt.A secure random number generator (Python's secrets or os.urandom) is used to generate a cryptographic salt, creating a random 16-byte salt.
3. The password and salt are used in PBKDF2-HMAC-SHA256 to generate a 256-bit AES key, following current OWASP password-storage advice for key derivation, with at least 480,000 iterations.
4. The contents of the file are then read and encrypted with AES-256-GCM, with a newly generated nonce; the result is a ciphertext and an authentication tag of 16 bytes.
5. The salt, nonce, authentication tag, and ciphertext are written to a new file (e.g., document.pdf.enc), and the original document is not modified except if the user chooses to securely delete it, and the file contains the salt, nonce, authentication tag and ciphertext in a known binary header format.

6.2 Decryption Workflow

1. The user chooses an encrypted file and types the password that was used to encrypt it.

2. Examines the salt, nonce and authentication tag contained in the file header and re-derives the AES key from the provided password using the stored salt.
3. The process of decryption and verification of the authentication tag occurs during AES-GCM operation, and if it fails — due to the fact that the password was incorrect or the file has been modified — the application fails and shows an error, not writing out anything.
4. On successful verification, the original file is reconstructed and saved in the name of the user.

6.3 Threat Model and Security Considerations

The attacker may get a full copy of the encrypted file (from a stolen laptop, leaked backup, etc) but not the password, and not be able to watch the application while in use. The idea behind this model is that using PBKDF2 to slow down the speed of the password-guessing attack offline, and AES-256-GCM to ensure confidentiality and integrity, should make it so that the encrypted file is hard to decrypt without the correct password and cannot be decrypted if the encrypted content is tampered with without the user's knowledge.

- Risk: passphrase length instead of password length (although not enforced, there's a UI password strength indicator to remind the user).Risk: weak/used passwords still remain the weakest link, despite the strength of the data encryption — mitigated by a password strength indicator in the UI and a generic suggestion to make passphrases longer than passwords.
- Risk: Avoiding nonce reuse under the same key would be a serious violation of the security guarantees of AES-GCM — this is the reason that a new key (e.g. a new salt) is used for each operation as well as a new random nonce.
- Risk: Incomplete or interrupted writes may corrupt output files — avoided by writing to a temporary file and renaming only if write is successful

7. Tools and Technologies

Component	Proposed Technology
Programming language	Python 3.10+
Graphical user interface	Tkinter (standard library)
Cryptography library	Python cryptography package (hazmat AEAD primitives)
Encryption method	AES-256-GCM (authenticated encryption)
Key derivation	PBKDF2-HMAC-SHA256, random salt, ≥480,000 iterations
Testing framework	pytest, for unit and round-trip integration tests

Version control	Git and GitHub
Documentation	Markdown, draw.io (for DFD/UML), Microsoft Word

8. Expected Outcome

The final product will feature a functioning GUI to encrypt and decrypt any file. There will be no information in the output that suggests the content of the original, and if an incorrect key were used to encrypt, or the encrypted content were altered, the attempt to decrypt it will fail spectacularly with a helpful error message and clear indication of the problem. Testing will validate, for various file types and sizes, that the files that are successfully decrypted are identical to the originals in size and content and that the performance of encryption/decryption is acceptable (less than 5 seconds for 100 MB files on typical consumer hardware). The final product will consist of the entire source code, a test suite documented, and a report that shows that the security decisions made in the implementation are based on well-defined standards in the field of cryptography

9. Testing and Evaluation Criteria

Test Category	**Criteria**
Correctness	Encrypt-then-decrypt round trip reproduces the original file exactly, verified via hash comparison (SHA-256).
Security	Incorrect password is rejected; modified ciphertext or tag fails authentication and produces no output file.
Usability	A test user unfamiliar with the tool can encrypt and decrypt a file without external instructions.
Performance	Encryption/decryption time is measured and reported across file sizes from 1 KB to 500 MB.
Robustness	Application handles missing files, permission errors, and interrupted operations without crashing or corrupting data.

10. Proposed Work Schedule

Stage	Deliverable	Main Activities
1	Project Proposal (5%)	Define the problem, aim, objectives, scope, and tools.
2	Literature Review (10%)	Study AES, PBKDF2, salts, nonces, AEAD, and known implementation pitfalls.
3	System Design (10%)	Prepare use case diagram, DFD, flowchart, and UML class/sequence diagrams.
4	Implementation (30%)	Develop the encryption engine, file-header format, GUI, and error handling.
5	Testing and Evaluation (15%)	Run functional, security, usability, and performance tests; record results.
6	Final Report (20%)	Compile research, design, implementation, results, limitations, and conclusion.
7	Presentation and Demonstration (10%)	Prepare slides and demonstrate the working application live.

11. Conclusion

This project will show that modern authenticated symmetric cryptography can be used properly to secure real files from disclosing and tampering. The proposed tool has incorporated AES-256-GCM, iterated Key Derivation Function (KDF) based on a password with proper salt, and every design decision has been based on the existing standards (FIPS 197, NIST SP 800-38D, NIST SP 800-132, and current OWASP guidance) to avoid common mistakes made by many amateur encryption tools, but at the same time is easy to use for a non-technical user. This will create both an application to include in the portfolio as well as an application which demonstrates an understanding of the fundamental cryptographic concepts covered in this course.

12. References (Preliminary)

National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)* (FIPS PUB 197). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.197

National Institute of Standards and Technology. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-38D

National Institute of Standards and Technology. (2010). *Recommendation for password-based key derivation* (NIST Special Publication 800-132). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-132

OWASP Foundation. (n.d.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Python Cryptographic Authority. (n.d.). *Cryptography package documentation*. https://cryptography.io/